

A Real-Time Traffic Digital Twin from Public CCTV: Vision-Based Detection, Map-Anchored Localization, and Self-Calibrating Speed Estimation

Minsoo Ahn
Department of Computer Science
Stony Brook University
minsoo.ahn@stonybrook.edu

Dahyun Kwon
Department of Computer Science
Stony Brook University
dahyun.kwon@stonybrook.edu

Abstract—We present a real-time traffic digital twin that converts live public CCTV feeds into an interactive, map-anchored view of road traffic. The system detects vehicles with YOLO26, tracks them with BoxMOT, and projects each pixel position to a geographic coordinate via a perspective homography. Because public CCTV cameras are uncalibrated, accurate ground-plane mapping and speed estimation are the central challenges. We address them with three mechanisms: (i) a two-stage curved transform that maps pixels along the true road centreline from the national node-link road network; (ii) a five-stage speed estimator with physics-based outlier rejection, sliding-window least-squares regression, and per-track exponential smoothing; and (iii) a per-camera scale factor updated online via an ITS-referenced calibration loop. The system additionally supports multi-camera background monitoring, congestion clustering, and a SQLite time-series history. We describe the architecture and key algorithms, and report latency, throughput, tracking stability, and localization accuracy over live streams.

Index Terms—digital twin, intelligent transportation systems, object detection, multi-object tracking, homography, perspective transform, speed estimation, real-time visualization.

I. INTRODUCTION

Cities expose thousands of public traffic cameras, but the raw feeds are difficult to interpret at scale: an operator watching a video grid cannot read off vehicle counts, speeds, or where congestion is forming on a map. A *digital twin*—a live, geo-referenced model of the road state—turns those feeds into actionable information. The technical obstacle is that public CCTV cameras are *uncalibrated*: their height, pitch, focal length, and exact heading are unknown, so projecting an image detection onto the ground plane (and therefore measuring distance and speed) is ill-posed without per-camera calibration.

This paper describes a system that ingests Korean ITS CCTV streams and produces a real-time map-anchored twin. Our contributions, from a systems and algorithms standpoint, are:

- **Map-anchored localization.** We snap each camera onto the national node-link road network, extract the road centreline, and map pixels along the *curve* of the road in two stages, rather than assuming a flat homographic plane. Satellite-referenced landmarks (Section V) show the pipeline places vehicles with 1.8–10.3 m lateral (on-road) accuracy; the curved stage constrains markers to

the mapped road shape on bends without degrading an already-accurate flat fit.

- **Robust speed estimation without ground truth.** A five-stage filter (duplicate skip, physics jump guard, sliding-window least-squares regression, per-track exponential smoothing with spike rejection, and scale correction) produces stable speeds despite detection jitter and tracker ID churn.
- **Per-camera scale correction.** Because absolute distance scale is unknowable from an uncalibrated camera, the system maintains a per-camera `speed_scale` that decouples geometric projection from absolute speed accuracy. An ITS-referenced online update loop with adaptive rate α_s (0.15 initially, decaying to 0.01) accumulates the factor across sessions; the current deployment has collected factors for 48 cameras (range 0.34–2.25), though multi-session operation is required for full convergence (see Section VI).
- **A complete, resilient pipeline.** Automatic lane-based calibration, manual 4-point calibration, dual-path inference (browser canvas and server-side HLS) over a shared detector, multi-camera background monitoring, congestion clustering, and a historical analytics store.

The rest of the paper is organized as follows. Section II reviews related work and positions this system relative to the state of the art. Section III presents the system architecture. Section IV details the algorithms. Section V reports the measurements. Section VI discusses limitations, and Section VII concludes.

II. RELATED WORK

Object detection and tracking. Modern traffic vision builds on real-time detectors. NMS-based architectures such as YOLOv8 [1] suppress redundant boxes through non-maximum suppression, but the threshold-sensitivity of NMS causes bounding-box positions to jitter between frames; this per-frame noise propagates directly into downstream speed estimates. To eliminate the source of positional instability, we adopt YOLO26 [2], an NMS-free end-to-end detector that produces more consistent bounding-box positions and simplifies the

TensorRT FP16 engine graph by removing the NMS layer [3], lowering both inference latency and its variance; YOLOv8 is retained as a legacy fallback. For tracking we use the BoxMOT framework [4], which exposes ByteTrack [5], BoT-SORT [6], and OC-SORT [7]; appearance re-identification uses OSNet [8]. Trackers without appearance ReID suffer ID switches under occlusion, which we mitigate with a lightweight position-based ID stabilizer.

Camera-to-ground mapping. Mapping image points to a ground plane is a homography problem [9], typically solved from four correspondences with OpenCV [10]. For *uncalibrated* cameras, vanishing-point and lane-geometry cues are commonly used to recover pose [11], but these techniques assume the ground plane is locally flat—an assumption that introduces lateral displacement errors of several metres on curved roads. Dubska et al. [11] achieve fully automatic calibration of roadside cameras from lane markings and vanishing points, but their model is restricted to a flat road plane; our pose solver extends this by additionally recovering camera height and pitch, and a second curved-centreline stage corrects residual lateral displacement on bends. To follow actual road geometry, our work introduces a road-centreline-following transform driven by a national road-network dataset [12], alongside a 4-point manual mode and a vanishing-point/lane auto-calibration path.

ITS dashboards and data. National ITS portals [13] publish CCTV streams and aggregated segment speeds, but typically as separate views; the aggregated speed is never fed back to correct the vision-derived estimate, leaving any geometric scale error unchecked. We bridge this gap by *fusing* the segment speed into the pipeline as an online calibration signal for per-camera scale correction. Congestion grouping uses density-based clustering [14] and convex hulls [15]; service levels follow the Highway Capacity Manual [16]. The visualization uses deck.gl [17] over MapLibre [18], served by FastAPI [19].

A. Positioning Relative to State of the Art

We position this work against representative monocular traffic-analysis systems along the dimensions most relevant to uncalibrated public-infrastructure deployment. Table I summarises the comparison; the discussion below groups the prior art into three families and states explicitly where our system agrees with, differs from, and is limited relative to each.

Closest setting: monocular CCTV speed estimation. The most directly comparable line of work estimates vehicle speed from a single uncalibrated surveillance camera. Sochor et al. [20] established the de-facto benchmark for this task, BrnoCompSpeed, with LIDAR-derived ground-truth speeds, and report sub-km/h mean error on full-HD footage of predominantly straight motorway segments. Revaud and Humenberger [21] target the same problem under realistic CCTV conditions—explicitly including low-resolution and non-straight roads—and, notably, evaluate on a dataset of Korean public CCTV streams with GPS-traced ground truth, the same infrastructure family this work builds on. Their calibration leverages the known 3D dimensions of vehicles; ours instead leverages the known 2D geometry of the road from the national

node-link network. The two priors are complementary: a 3D-vehicle prior recovers metric scale per detection, whereas our centreline prior recovers road-following position over an extended corridor. We note that although the Revaud data *includes* non-straight roads, their method estimates speed without anchoring each vehicle to the road’s mapped curvature; it is robust *on* curved scenes but does not constrain positions *to* the curve, which is why we mark it partial (Δ) rather than full in the “Curved road” column of Table I. FARSEC [22] subsequently provided a reproducible pipeline evaluated on both BrnoCompSpeed and the Revaud CCTV set, underscoring that these two datasets are the accepted reference points for the task.

We emphasise an honest distinction. These methods report quantitative speed accuracy against instrumented ground truth, which our present evaluation does not: our speed figures (Table V) confirm plausible output rather than benchmark accuracy, and the per-camera `speed_scale` factors remain unconverged at the time of writing. We do, however, validate *localization* quantitatively against satellite-referenced landmarks (Section V), which is the dimension we claim as the contribution. Establishing comparable speed accuracy on the Revaud Korean-CCTV set or BrnoCompSpeed is the natural next step and is left as future work; the contribution claimed here is the map-anchored localization and self-calibrating pipeline, not a new speed-accuracy record.

Automatic calibration from road geometry. A second family recovers camera pose automatically from lane markings and vanishing points. Dubska et al. [11] achieve fully automatic calibration of roadside cameras from these cues, and Sochor et al. [20] refine scene scale by aligning 3D vehicle bounding boxes, reducing scale error substantially over earlier automatic methods. Both, however, model the road as a single flat plane. Our pose solver (`camera_pose.solve_pose`) follows the same vanishing-point/lane-edge strategy but additionally recovers camera height and pitch against a known road width, and—critically—a second centreline-following stage (Algorithm 1) corrects the residual lateral displacement that a flat-plane model leaves on bends. This is the dimension labelled “Curved road” in Table I, where prior monocular methods are marked unsupported.

Dedicated sensors and on-vehicle GPS. A third family sidesteps calibration entirely. Inductive-loop detectors, roadside radar, and on-vehicle GPS loggers measure speed and position directly, but require either physical roadside installation or instrumentation of the vehicles themselves. Our system adds no hardware to the road or to vehicles, re-purposing cameras that road authorities already operate; the trade-off is that all geometry is inferred and therefore bounded by the quality of the map and the perspective projection (Section VI).

Generic detection/tracking pipelines. Finally, open-source YOLO+BoxMOT pipelines provide bounding boxes and track IDs but no geographic anchoring, calibrated speed, or support for uncalibrated cameras out of the box. Our system integrates these components and adds the localization, calibration, and self-correction layers that turn raw tracks into a map-anchored

TABLE I

COMPARISON WITH REPRESENTATIVE TRAFFIC-MONITORING APPROACHES. ✓ = FULLY SUPPORTED; × = NOT SUPPORTED; △ = PARTIALLY SUPPORTED; — = NOT APPLICABLE.

Method	No road HW	No vehicle GPS	Auto- calib	Curved road	Speed est.	Real- time	Multi- cam	Open- source
Loop / Radar sensors	×	✓	—	—	✓	✓	△	×
On-vehicle GPS	✓	×	—	✓	✓	✓	✓	△
YOLO + flat homography	✓	✓	×	×	△	✓	×	✓
Dubská et al. [11]	✓	✓	×	×	✓	✓	×	×
Sochor et al. [20]	✓	✓	✓	×	✓	✓	×	✓
Revaud & Humenberger [21]	✓	✓	✓	△	✓	✓	×	✓
This work	✓	✓	✓	✓	✓	✓	✓	✓

twin. Among the compared systems, ours is the only one supporting the full set of columns in Table I; we nonetheless note that this breadth reflects a systems-integration contribution rather than a single-metric advance, and that the per-axis novelty (curved localization, ITS-referenced scale) is what we claim as new.

III. SYSTEM DESIGN AND ARCHITECTURE

The backend (FastAPI/Uvicorn) and a React/deck.gl frontend communicate over REST and WebSocket. Two inference paths can produce per-frame analytics and *share a single detector*; a connection counter coordinates them so that the multi-object tracker is never driven by two interleaved frame streams (Fig. 1). A two-lock scheme inside the detector (`_state_lock` for tracker state, `_lock` for GPU inference) prevents data races between thread-pool threads, and module-level write locks in `main.py` and `roi_manager.py` atomize all JSON configuration read-modify-write operations.

Because running a separate detector per connection would multiply GPU memory use and violate the shared-tracker invariant, both paths feed a single shared detector; the connection counter ensures only one path drives the multi-object tracker at a time. The two paths are designed for complementary operating conditions.

Browser path (/ws/detect). When an operator opens a camera view, the frontend plays the HLS stream through a CORS proxy, captures canvas frames (JPEG, ≤ 640 px, ≈ 33 ms interval, at most two in flight), and sends them to the backend, which runs detection, tracking, localization, and analytics, then returns an annotated frame and broadcasts the analytics JSON to all clients.

Server path (live_loop). Without an active browser client, the backend reads HLS directly with OpenCV/FFmpeg and runs the same pipeline. It is gated twice: it skips tracking while a browser detection client is active (to protect the shared tracker), and it skips all GPU work when no one is watching (the frontend reports tab visibility). Before opening any HLS URL the detector pre-resolves HTTP 302 redirects (`_resolve_hls_url`), so FFmpeg receives a direct endpoint and does not silently fail on delivery-layer redirect chains.

Beyond the two inference paths, sustained operation requires housekeeping that neither path handles directly: HLS access tokens expire on a 30-minute cycle, the ITS speed reference

drifts unless polled, and the congestion map must be periodically refreshed. **Background services.** Periodic loops therefore refresh expiring HLS tokens (30 min), poll the ITS segment speed for self-calibration (5 min), and snapshot all monitored cameras into a SQLite history while recomputing congestion clusters (30 s). To prevent the FOV polygon from flickering during auto-calibration, a slow EMA filter stabilizes near/far distance and road-width estimates, holding the initial accepted value fixed until sufficient samples accumulate. ROI edits trigger an immediate GPS ring recomputation and broadcast to keep the map view synchronized. The resulting map view is shown in Fig. 2.

IV. IMPLEMENTATION

A. Detection and Tracking

The detector loads the fastest available backend—TensorRT FP16 engine, then ONNX Runtime, then PyTorch—and exports `.pt`→`.engine` on first run when a GPU is present. A tracker tier is chosen by GPU memory: ByteTrack (CPU-only, no CUDA), OC-SORT (low VRAM ≥ 3.5 GB, occlusion-robust, no ReID), or BoT-SORT/DeepOC-SORT with OSNet appearance ReID on larger GPUs (≥ 6 GB and ≥ 10 GB respectively). Without appearance ReID, two failure modes degrade practical tracking quality: detectors occasionally fire more than once on the same vehicle, inflating vehicle counts, and track identities reset after brief occlusions rather than recovering the original ID, causing discontinuous trails on the map. To suppress both effects, the non-ReID tiers include two lightweight robustness layers. `_dedup_tracks` merges duplicate boxes (IoU > 0.3 or centre distance < 40 px, keeping the older ID), while an `IDStabilizer` re-binds a vehicle’s previous ID after a brief miss using nearest-centre matching (≤ 80 px) with a two-pass scheme and stale-mapping purge so display IDs stay stable and compact. Fig. 3 shows the detection and tracking overlay.

B. Map-Anchored Localization

On camera switch we query the national node-link network: we snap the camera position perpendicularly onto the nearest road polyline, extract ± 150 m of centreline, and—because each direction is stored as a separate one-way link—we locate the reverse carriageway and average the two snaps (when 2–40 m apart) to recover the true road centre. The chosen road’s speed limit, lane count, and width (lanes $\times 2 \times$ lane-width) are derived from NodeLink and attached to the camera.

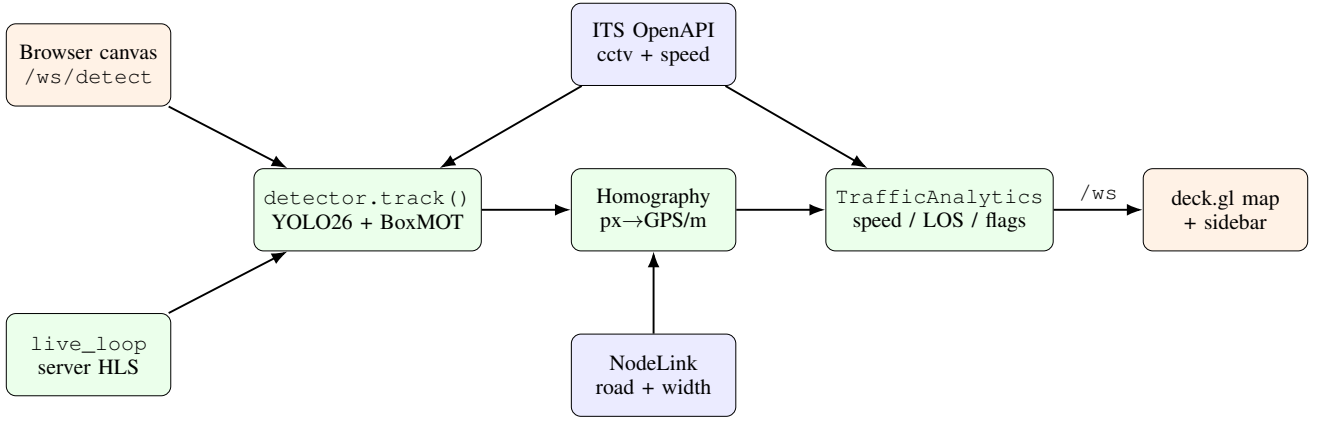


Fig. 1. System architecture. The browser path (`/ws/detect`) and the server path (`live_loop`) share one detector; a counter prevents concurrent tracker calls. External data (ITS CCTV/speed, NodeLink) feeds localization and self-calibration.

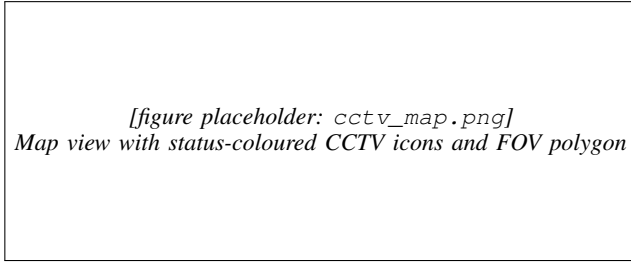


Fig. 2. Map view: public CCTV cameras as status-coloured icons on the vector map; the selected camera shows its field-of-view polygon and the located vehicles.

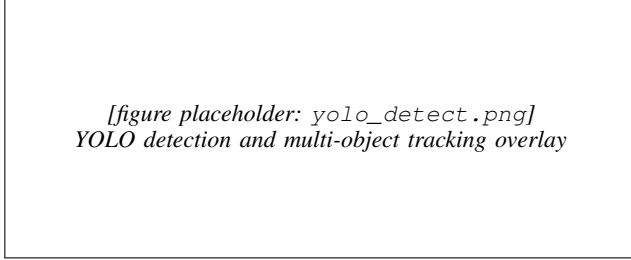


Fig. 3. Detection and tracking on a live CCTV frame: bounding boxes, vehicle class, and stabilized track IDs.

A single flat homography maps pixels to a plane tangent to the camera’s snap point; on curved roads, this plane diverges from the true centreline and displaces vehicle markers laterally by several metres. To follow the actual road geometry, we instead introduce the *two-stage curved transform* (Algorithm 1). The metric homography H_m is derived from the same four image–GPS correspondences as the GPS homography, but maps pixel coordinates to local ENU metres rather than geographic coordinates, providing a metric workspace in which arc-length displacement can be computed accurately. Stage one maps the pixel to local metres via H_m and converts the result to a geographic coordinate via the snap-point anchor (ENUTOGPS); stage two projects that coordinate perpendicularly onto the centreline polyline (PERPPROJECT, Algorithm 2), obtaining the true arc length and a signed lateral offset directly from the

nearest point on the curve, then re-applies the lateral offset perpendicular to the local road bearing at that arc. This replaces the earlier fixed-bearing decomposition ($d_{\parallel} = dx \sin b + dy \cos b$), which under-measured the true arc length on bends and folded the road’s own curvature into the lateral term as a spurious offset; the perpendicular projection follows the bend and eliminates that artifact.

Algorithm 1 Two-stage curved pixel→GPS transform (perpendicular-projection)

Require: pixel (u, v) ; metric homography H_m ; snap point p_m (ENU m); snap GPS (ϕ_0, λ_0) ; centreline polyline $C = \{(\phi_i, \lambda_i)\}$ with cumulative arc lengths $\{\ell_i\}$

- 1: $(x_m, y_m) \leftarrow H_m \cdot (u, v)$ ▷ pixel → ENU metres
- 2: $(dx, dy) \leftarrow (x_m, y_m) - p_m$ ▷ ENU offset from snap
- 3: $(\phi_e, \lambda_e) \leftarrow \text{ENUTOGPS}(\phi_0, \lambda_0, dx, dy)$
- 4: $(arc, d_{\perp}) \leftarrow \text{PERPPROJECT}((\phi_e, \lambda_e), C)$ ▷ nearest point on centreline
- 5: $(\phi_c, \lambda_c) \leftarrow \text{INTERPCENTERLINE}(arc)$
- 6: $b_{\ell} \leftarrow \text{ROADBEARINGAT}(arc)$
- 7: **apply** $d_{\perp}$ perpendicular to b_{ℓ} at (ϕ_c, λ_c)
- 8: **return** (ϕ, λ)

When no manual calibration exists, an automatic procedure estimates a homography from one frame: Canny edges in the lower image, Hough line filtering, multi-row sampling of left/right road edges, least-squares edge fits, vanishing-point intersection, a pitch/height estimate from a pinhole model, and a trapezoid-to-GPS homography. Direction (which way the camera looks along the road) is resolved by matching the *image* road-curve sign against the *map* curve sign, falling back to a vanishing-point heading test. A manual 4-point mode remains available for maximum accuracy. Fig. 4 shows an automatic calibration result.

The primary auto-calibration path is a road-model *pose solver* (`camera_pose.solve_pose`): the sampled lane edges, vanishing point, and NodeLink road width are fitted to a four-parameter pinhole-camera model (H , pitch, yaw,

Algorithm 2 PERPPROJECT: perpendicular projection onto centreline

Require: query point $q = (\phi, \lambda)$; centreline $C = \{(\phi_i, \lambda_i)\}$ with arc lengths $\{\ell_i\}$

- 1: $d^* \leftarrow \infty$
- 2: **for** each segment $(\phi_i, \lambda_i) \rightarrow (\phi_{i+1}, \lambda_{i+1})$ **do**
- 3: $\vec{b} \leftarrow$ segment vector (local ENU metres)
- 4: $\vec{p} \leftarrow q - (\phi_i, \lambda_i)$ (local ENU metres)
- 5: $t \leftarrow \text{clamp}(\langle \vec{p}, \vec{b} \rangle / \|\vec{b}\|^2, 0, 1)$
- 6: $s \leftarrow t\vec{b}$ ▷ foot of perpendicular
- 7: $d \leftarrow \|\vec{p} - s\|$
- 8: **if** $d < d^*$ **then**
- 9: $d^* \leftarrow d$
- 10: $\text{arc} \leftarrow \ell_i + t \|\vec{b}\|$
- 11: $d_\perp \leftarrow \text{sign}(b_x p_y - b_y p_x) \cdot d$ ▷ signed lateral
- 12: **end if**
- 13: **end for**
- 14: **return** $(\text{arc}, d_\perp)$

[figure placeholder: auto_calib.png]
Map view with FOV footprint after automatic calibration

Fig. 4. Map view showing the estimated field-of-view footprint after automatic calibration: the cyan corridor follows the road centreline extracted from the NodeLink dataset.

lateral offset x_0) via `scipy.optimize.least_squares` (soft-L1 loss). A per-row reprojection residual gates the output (< 8 px); on success the solved pose is converted to four image-GPS corner pairs and used to build both homographies. The pose is then persisted per camera to `camera_pose.json` so each session refines the prior, with a cold-start fallback chain (saved prior $\rightarrow$ vehicle-bbox rough pose $\rightarrow$ GPS-grid). A complementary `vehicle_calib.json` accumulates a per-camera linear pixel-to-distance model ($d = B \cdot y_{px} + C$, implied vanishing-point $y_v = -C/B$) fitted from observed vehicle bounding-box heights; it is restored on session start and updated online whenever sufficient samples accumulate. This provides a second, independent distance estimate—derived from the known apparent size of vehicles rather than from road geometry—that serves as a *self-consistency check* on the geometric homography. We use it as an auxiliary signal rather than a primary calibration: it does not replace the homography, but a large disagreement between the two flags an unreliable geometric fit. We do not claim this size-based scale as novel—recovering scene scale from known vehicle dimensions is established [20], [21]; here it functions only as a lightweight, training-free cross-check. The trapezoid heuristic above serves only as the fallback when the residual exceeds the quality gate or lane detection fails.

Algorithm 3 Per-track speed estimation (analytics._speed)

Require: window W , prior EMA v_e , $\sigma \leftarrow \text{speed_scale}$, $\kappa \leftarrow \text{depth_corr}(y_{\text{bbox}})$

- 1: **if** duplicate position **then** skip ▷ dedup
- 2: **if** $\text{step} > (V_{\text{max}}/3.6\sigma) dt \cdot 3.0$ **then** clear W ▷ teleport reset
- 3: **else if** $\text{step} > (V_{\text{max}}/3.6\sigma) dt \cdot 1.5$ **then** drop sample, keep W ▷ jump guard
- 4: **if** $dt > 2$ s **then** clear W ▷ track gap
- 5: fit $\hat{v}$ by OLS on W (0.7 s); **if** $\text{disp}(W) < 0.5$ m **then** $\hat{v} \leftarrow 0$ ▷ OLS
- 6: $v_e \leftarrow (1-\alpha)v_e + \alpha\hat{v}\kappa\sigma$, $\alpha=0.35$; reject spike; decay on stop ▷ EMA
- 7: **output** v_e ; $\text{is_speeding} \leftarrow \text{reliable} \wedge v_e > 1.10 \cdot \text{limit}$ ▷ output

After GPS coordinates are assigned, a two-step post-processing pass reduces visual noise. Per-track EMA smoothing eliminates lateral jitter while preserving along-road motion; a jump threshold resets the filter after occlusion gaps to handle track re-appearances correctly. A lane-offset pass then laterally separates inbound and outbound vehicles perpendicular to the local road bearing, placing each direction on its correct side of the road centreline for Korean right-hand traffic.

C. Speed Estimation

Speed is distance over time, and both terms are error-prone in this setting. Homography-derived positions carry growing noise toward the far field, where small pixel displacements correspond to large ground-plane uncertainties. HLS streams compound the problem: segments are buffered and retransmitted with variable delivery latency, so consecutive frame-arrival times reflect network scheduling rather than true capture cadence, making wall-clock timing unreliable. Rather than treat these two error sources independently, we address them jointly: a monotonic clock derived from stream presentation timestamps (falling back to wall-clock) decouples timing from delivery jitter, while the five-stage estimator of Algorithm 3 filters the resulting distance-over-time signal using the constants in Table II.

All thresholds in Table II were empirically tuned on live ITS streams; sensitivity analysis is left as future work.

Two additional reliability mechanisms reduce systematic bias in the speed estimate. First, a depth-varying correction factor κ is computed from a per-camera vehicle apparent-size model (a linear fit of inverse apparent bbox width versus vertical pixel position), compensating for the depth-dependent projection error of the homography without introducing frame-to-frame position discontinuities. Second, vehicles whose GPS position lies more than 100 m from the camera snap point (`SPEED_TRUST_MAX_DEPTH_M`) are flagged `speed_reliable=False`: their speed is still displayed, but excluded from the frame average, the ITS calibration input, and speeding

TABLE II
KEY SPEED/ANALYTICS CONSTANTS (CONFIG.PY).

Parameter	Value
Speed window	0.7 s ($W=\text{round}(0.7s \times \text{FPS})$)
EMA α	0.35
Jitter threshold	0.5 m
Min speed (else 0)	5 km/h
Max plausible $V_{\max}$	180 km/h
GC grace	30 frames
Bottleneck dwell	150 frames (≈ 5 s)
Parked dwell	300 frames (≈ 10 s)
Speeding threshold	$v_e > 1.10 \times \text{limit}$
LOS thresholds A–E	$\leq 3, 6, 9, 12, 15$ (else F)
Background busy	> 6 vehicles
Background congested	> 14 vehicles
Cluster ϵ	500 m (haversine)

alerts, preventing far-field pixel noise from biasing aggregate statistics.

D. Per-Camera Scale Correction

The absolute distance scale of an uncalibrated camera is unknowable from geometry alone, so a residual scale error persists after geometric calibration. The system maintains a per-camera `speed_scale` factor that multiplies the geometric speed estimate (Algorithm 3, line 6) and is updated online using the ITS official segment speed as a soft reference. The ITS signal carries inherent uncertainty—aggregated over ≈ 1 km segments at 5-minute intervals, the reported direction may not match the camera’s field of view—so the loop adapts slowly:

$$\text{target} = \text{scale} \cdot \frac{v_{\text{ITS}}}{v_{\text{ours}}}, \quad \text{scale} \leftarrow (1 - \alpha_s) \text{scale} + \alpha_s \text{target}, \quad (1)$$

with an adaptive α_s schedule and a per-update $\pm 10\%$ step clamp in addition to the absolute $[0.3, 5.0]$ bounds. For cameras without a previously saved factor (first session), α_s decreases from 0.15 (updates 1–2) through 0.05 (updates 3–4) to 0.01 thereafter, enabling fast initial convergence while damping later oscillation. For cameras whose factor was restored from `speed_scale.json`, α_s is fixed at 0.01 to preserve the accumulated calibration. The calibration uses a 10-minute rolling window of continuous, per-frame speed observations, requiring at least 50 frames with valid vehicle detections to trigger the correction, and a coefficient-of-variation guard (≤ 0.4) to skip volatile traffic. The factor persists per camera and accumulates across sessions without resetting the geometric calibration (Section VI).

E. Aggregation, Congestion, and History

Assigning inbound versus outbound direction from a single frame’s pixel displacement is unreliable when a vehicle moves slowly, stops briefly, or travels nearly perpendicular to the camera view—the displacement vector is then too short or misaligned to determine sign robustly. To accumulate directional evidence across frames, each vehicle’s inter-frame displacement is projected onto the road bearing axis and smoothed with an EMA deadzone filter that commits a direction

[figure placeholder: *congestion_map.png*]
Congestion overlay: CCTV icons recoloured by status

Fig. 5. Camera-level congestion: monitored cameras are recoloured (normal/busy/congested) and adjacent congested cameras are grouped into a severity-shaded region.

only when the smoothed component exceeds an ambiguity threshold. When road bearing is unavailable, a horizontal LineZone crossing serves as a fallback and also provides In/Out counts. Dwell counting flags bottlenecks and parked vehicles (the latter remembered by pixel position so they survive ID changes), and a Level-of-Service grade is assigned from the active vehicle count. Background monitoring polls additional cameras every 8 s with detection only; cameras with more than 6 vehicles are marked *busy* and those with more than 14 are marked *congested*; such cameras are then clustered with a haversine density-based scan ($\epsilon=500$ m) and outlined with a convex hull. A WAL-mode SQLite store records 30 s snapshots (14-day retention) and serves bucketed time series, peak-time detection, and CSV export. Fig. 5 shows the congestion overlay.

F. Visualization

The deck.gl map layers congestion polygons, vehicle trails, the FOV polygon, status-coloured camera icons, and vehicle markers. The FOV polygon uses one of three strategies in priority order: (1) the manual calibration ring (`calibGpsRing`), the exact four GPS corners entered by the user; (2) a curved polygon computed by walking the road centreline and laterally offsetting by half the road width (`computeRoadCorridorPolygon`), so the footprint follows real road curvature rather than a flat projection; or (3) a straight-sided rectangle derived from near/far distance and road width (`computeCalibPolygon`), falling back to a default 70° trapezoid (`computeFovPolygon`) when uncalibrated.

The sidebar’s vehicle list provides *direction-filter tabs* (All / Inbound / Outbound) that filter the displayed rows by the movement-vector-based direction; the per-direction vehicle count is shown in each tab badge, and the min/avg/max speed summary is recomputed from the filtered set. Two collapsible cards (Auto Calibration Estimate, ITS Speed Comparison) include an info button that opens a modal explanation of each metric, adapting to the selected map theme.

V. EVALUATION

Measurements are produced from the real pipeline in two ways. While the system runs (`make dev`), an in-process collector records per-frame stage latency, tracking, speed, and detection statistics and flushes `eval_*.csv` and `eval_summary.json` every 30 s (also on

TABLE III

INFERENCE BACKEND COMPARISON (OC-SORT TRACKER FIXED, 1,000 FRAMES, RTX 4070 LAPTOP, NATIONAL ROUTE 6 NAM YANGJU). TRACK STAGE INCLUDES YOLO + BOXMOT END-TO-END. RESULTS FROM THE ARCHIVED EVALUATION RUN STORED IN THE REPOSITORY.

Backend	Mean (ms)	p95 (ms)	FPS
TensorRT FP16	6.7	8.7	147.6
PyTorch (CUDA)	8.4	11.6	117.6
ONNX Runtime	13.9	16.8	71.5

TABLE IV

TRACKER COMPARISON (TENSORRT FP16 BACKEND FIXED, 1,000 FRAMES, NATIONAL ROUTE 6 NAM YANGJU, ARCHIVED RUN). BoT-SORT OVERHEAD IS FROM OSNET APPEARANCE REID; BYTETRACK SHOWS MORE ID CREATION EVENTS DUE TO THE ABSENCE OF RE-IDENTIFICATION.

Tracker	Mean (ms)	p95 (ms)	FPS	IDs	ID ev.	Life (fr.)
ByteTrack	6.6	7.8	150.9	27	286	35.6
OC-SORT	6.6	8.9	150.2	8	18	80.0
BoT-SORT	10.7	18.5	93.4	5	16	132.0

IDs = unique track IDs; ID ev. = ID creation events; Life = mean lifetime.

demand via `GET /eval/report`); an offline harness (`backend/evaluate.py`) reproduces the same metrics over a live HLS source for controlled benchmarks. The system auto-selects the inference backend (TensorRT $\rightarrow$ ONNX Runtime $\rightarrow$ PyTorch) and tracker tier (ByteTrack/OC-SORT/BoT-SORT/DeepOC-SORT by GPU VRAM).

Hardware. All timing figures are from a single machine: NVIDIA GeForce RTX 4070 Laptop GPU (8 GB VRAM, compute capability 8.9), driver 610.47, TensorRT 10.16, ONNX Runtime 1.25, Windows 11. Absolute latency and FPS are hardware-specific; the relative ordering across tiers and backends is the transferable result.

Run conditions. Model YOLO26m, source: National Route 6 Namyangju Dongmakgol Intersection (live ITS HLS, nighttime), 1,000 frames per run (20-frame warmup excluded). Backend and tracker are varied across runs as shown in Tables III and IV; all other conditions are held constant. Each table is an independent measurement run, so the shared TensorRT + OC-SORT operating point differs slightly across Tables III–IV, and the tracking-stability counts vary with the traffic present at each capture time. Speed and vehicle-count figures in Table V are from the TensorRT + OC-SORT run and reflect nighttime traffic at this specific location; they are included to confirm the pipeline produces plausible output, not as ground-truth benchmarks.

TensorRT FP16 achieves the lowest latency (6.7 ms mean), $2.1\times$ faster than ONNX Runtime and $1.25\times$ faster than PyTorch on the same GPU. The evaluation harness additionally breaks end-to-end time into pipeline stages; the homography transform and analytics stages each contribute under 0.5 ms (median 0.14 ms and 0.40 ms respectively), confirming that the localization and speed-estimation layers add negligible overhead to the dominant tracking stage. ByteTrack and OC-SORT are iso-latency (≈ 6.6 ms), but OC-SORT’s occlusion-centric cost function reduces ID creation events from 286 to

TABLE V

TRACKING STABILITY, SPEED DISTRIBUTION, AND DETECTION BREAKDOWN (TENSORRT + OC-SORT RUN, 1,000 FRAMES, NIGHTTIME).

Metric	Value
Frames processed	1,000
Unique tracks	12
Track creation events (re-entries + re-acquisitions)	58 ($4.83\times$ unique tracks)
Mean track lifetime (frames)	124.2
Median track lifetime (frames)	131.5
Max track lifetime (frames)	229
Vehicle speed – mean (km/h)	90.9
Vehicle speed – median (km/h)	78.2
Vehicle speed – p95 (km/h)	174.2 [†]
% moving observations	79.6 %
Total speed observations	1,186 of 1,490 detections [‡]
Detections – car	1,424
Detections – truck	66

[†]Far-field outliers; median 78.2 km/h is representative.

[‡]Speed requires $\geq W$ track-history frames; single-frame detections excluded.

18 and more than doubles mean track lifetime ($35.6 \rightarrow 80.0$ frames); BoT-SORT adds OSNet ReID, further reducing ID events to 16 and improving lifetime to 132.0 frames at the cost of $1.6\times$ higher latency (10.7 ms) and a wider p95 (18.5 ms vs 8.9 ms).

Per-camera scale calibration. Because no per-vehicle ground truth exists for public CCTV, the system maintains a per-camera `speed_scale` updated online via the ITS-referenced loop. Table VI summarises the factors accumulated across all 48 cameras whose streams have been monitored (`speed_scale.json`). These factors are still *accumulating* rather than failed: convergence requires sustained multi-session operation per camera (Section VI), and the snapshot reported here captures a single monitoring pass over many cameras rather than repeated visits to any one. Crucially, the system produces map-anchored output throughout this accumulation—the scale factor refines the speed estimate but is not a precondition for localization. The majority (62.5 %) fall within $\pm 20\%$ of 1.0, confirming that the geometric calibration is roughly correct for most cameras; the left tail (20.8 % below 0.8) reflects cameras with steep pitch or far-field observation zones that compress apparent displacement, while the right tail (8.3 % above 1.2) corresponds to wide-angle or low-mounted cameras that exaggerate it.

Satellite-referenced localization accuracy. To measure localization directly—without instrumented vehicles—we adopt a landmark-based protocol in the spirit of Revaud and Humenberger’s `evaluate_homographies` [21], which scores the homography against fixed ground-truth references and so isolates geometry from detector and tracker noise. Rather than GPS-tracked cars, we use fixed road landmarks (lane-dash ends, painted markings, structural points) whose true coordinates are read from high-resolution aerial imagery; for each landmark we query the running backend in its current (pose-calibrated)

TABLE VI

DISTRIBUTION OF LEARNED `SPEED_SCALE` FACTORS ACROSS 48 CAMERAS (`SPEED_SCALE.JSON`). VALUES BELOW 1.0 INDICATE GEOMETRIC UNDERESTIMATION OF DISPLACEMENT (COMMON WHEN THE CAMERA AXIS IS OBLIQUE TO THE ROAD); VALUES ABOVE 1.0 INDICATE OVERESTIMATION. FACTORS ARE STILL ACCUMULATING TOWARD CONVERGENCE AT THIS SNAPSHOT (SINGLE-PASS MONITORING; SEE SECTION VI).

Range	Cameras	Share
< 0.5	4	8.3 %
0.5–0.8	10	20.8 %
0.8–1.2	30	62.5 %
> 1.2	4	8.3 %
Min / Max: 0.34 / 2.25		
Mean / Median: 0.92 / 0.91		

state and compare its `pixel_to_gps` output against ground truth, decomposing the error into along-road (longitudinal) and lateral (off-road) components—the two modes the paper separates. The landmarks are independent of any calibration input, so the figures reflect generalization, not fit.

Table VII reports two pose-calibrated cameras on national routes, each scored under the flat homography and the curved-centreline transform. Two facts stand out. First, the system places vehicles with low *lateral* error—1.8 m on the better-conditioned camera (Rte. 6, 5 px reprojection residual) and 10.3 m on the more obliquely-mounted one (Rte. 3, 6.4 px)—confirming that the map-anchored geometry puts vehicles on the correct part of the carriageway. Second, the lateral and longitudinal components behave exactly as the orthogonality analysis in Section VI predicts: the longitudinal error is far larger (29–40 m), set by the residual focal-length scale uncertainty, while the lateral error is small and is the only component the curved transform can act on.

On these well-calibrated cameras the curved transform is *neutral*: it changes lateral error by under 0.2 m, because when the metric homography is accurate the flat plane already recovers ground positions correctly on a planar curve (every point of the curve lies on the fitted $z=0$ plane). The curved transform is therefore best understood not as a uniform accuracy improvement but as a lateral-conformance safeguard that constrains markers and the FOV polygon to the road’s mapped shape without degrading an already-accurate flat fit—and it engages meaningfully only when that flat fit is weak. The dominant residual on both cameras is longitudinal, which no geometric transform corrects; it is the target of the `speed_scale` loop (Section VI).

VI. DISCUSSION AND LIMITATIONS

The homography is accurate inside the calibration quadrilateral and extrapolates with growing error toward the far field; we mitigate this with the dual-matrix calibration, the regression window, and lane-based auto-calibration, but a residual error remains on poorly observed cameras. Automatic calibration assumes a focal length of $1.2\times$ image height (unknown intrinsics), giving a $\pm 15\text{--}20\%$ scale uncertainty per

TABLE VII

SATELLITE-REFERENCED LOCALIZATION ERROR (METRES) ON TWO POSE-CALIBRATED CAMERAS, FLAT HOMOGRAPHY VS. CURVED-CENTRELINE TRANSFORM, DECOMPOSED INTO LATERAL (OFF-ROAD) AND LONGITUDINAL (ALONG-ROAD) COMPONENTS. LANDMARKS ARE INDEPENDENT OF CALIBRATION POINTS; GROUND TRUTH FROM AERIAL IMAGERY, QUERIED AGAINST THE LIVE BACKEND. THE ERROR IS DOMINATED BY THE LONGITUDINAL COMPONENT (FOCAL-SCALE UNCERTAINTY, SECTION VI); THE CURVED TRANSFORM ACTS ONLY ON THE SMALL LATERAL COMPONENT AND IS NEUTRAL HERE BECAUSE THE FLAT FIT IS ALREADY ACCURATE. LANDMARK COORDINATES AND THE QUERY SCRIPT ARE TO BE DEPOSITED IN THE PUBLIC REPOSITORY TO SUPPORT REPRODUCIBILITY.

Camera (calib. residual)	Method	Lateral	Along	Total
Rte. 3 (6.4 px)	flat	10.3	40.3	41.6
	curved	10.1	40.4	42.4
Rte. 6 (5 px)	flat	1.8	29.2	29.3
	curved	1.8	29.1	29.2

Lateral / Along = MAE of off-road / along-road error; 4–5 landmarks per camera.

FoV mismatch. It also fails when lane markings are not visible (night, rain, dense traffic), falling back to an approximate grid. Road width is inferred from lane count and assumed bidirectional, so one-way roads are over-wide. The nearest node-link road is occasionally the wrong one (e.g., a camera near both a national route and an expressway), which displaces the FOV polygon and vehicle GPS; the CCTV-name road-hint reduces but does not eliminate this. Finally, congestion is clustered at the camera level because background cameras expose only counts, not per-vehicle speeds.

Error structure of curve localization, and the roles of the two corrections. The two-stage centreline transform (Algorithm 1) and the per-camera `speed_scale` loop (Section IV-D) are easily conflated as a single “accuracy” mechanism. A controlled synthetic analysis—projecting vehicles onto a circular-arc centreline of known curvature with exact ground-truth positions and recovering them through both the flat homography and Algorithm 1—yields a more precise account of what each component does.¹

Two findings emerge. First, when the metric homography H_m is geometrically accurate, the flat homography already recovers ground positions correctly on a planar curve, since every point of the curve lies on the same $z=0$ plane the homography was fitted to; in this regime the centreline transform changes accuracy negligibly. The displacement that matters in practice comes instead from the focal-length assumption in the automatic calibration path ($f_y = 1.2h$, a $\pm 15\text{--}20\%$ FoV-dependent scale error), and it falls almost entirely in the *longitudinal* (depth) direction rather than laterally. Second, the original fixed-bearing decomposition in Algorithm 1 constituted a coordinate artifact: resolving the

¹Synthetic conditions: pinhole camera at 10 m height, 18° downward pitch, focal length $f = 864$ px ($1.2\times$ a 720 px frame height), snap point 10 m ahead of the camera; circular-arc roads of radius $R \in \{80, 120, 200, 300, 500, 1000, \infty\}$ m over a 160 m corridor; vehicles placed with up to ± 3.5 m lane offset and 1.5 px detection noise. Errors are exact since ground-truth positions are known by construction.

ENU displacement against the snap point’s single fixed bearing under-measured the true arc length on a bend and folded the road’s own curvature into the lateral term as a spurious offset, displacing markers by tens of metres at small radii in simulation. Replacing that decomposition with the perpendicular projection (PERPPROJECT, Algorithm 2)—which follows the bend and recovers the true arc length—removes the artifact and reduces the simulated off-road lateral error to below one metre across the tested curvature range, while leaving the straight-road case unchanged.

These results show that the lateral and longitudinal error modes are geometrically orthogonal and require different corrections. The satellite-referenced measurements in Section V (Table VII) confirm this structure on real cameras: on both pose-calibrated cameras the error is dominated by the longitudinal component (29–40 m, set by focal-scale uncertainty), while the lateral component is small (1.8–10.3 m) and is the only axis the curved transform alters—changing it by under 0.2 m when the flat fit is already accurate. The real data thus supports the orthogonality claim and the role assignment (geometry for lateral placement, the `speed_scale` loop for longitudinal scale), while showing that the curved transform is a safeguard that does not degrade a good flat fit rather than a uniform accuracy gain. The centreline transform addresses only the *lateral* component: it constrains vehicle markers and the FOV polygon to the road’s true shape on bends, but it cannot recover absolute longitudinal scale, since the along-road distance inherits whatever scale error the homography carries. That residual scale error is absorbed instead by the ITS-referenced `speed_scale` loop, which converges a per-camera multiplicative factor against the official segment speed. The two mechanisms are therefore complementary rather than redundant: the geometry fixes *where on the road* a vehicle is placed, and the ITS loop fixes *how far* the geometry believes it has travelled. This separation also disciplines evaluation: lateral road-alignment can be judged geometrically (markers either lie on the carriageway or they do not), whereas longitudinal accuracy is only meaningful against a speed or distance reference.

The ITS-based self-calibration loop uses an adaptive learning rate (0.15 on the first two updates, decaying to 0.01 in later updates, every 5 min) to treat the ITS segment speed as an imprecise soft reference, since ITS figures reflect 1-km road-segment averages that may not align with the camera’s exact observation zone or direction. The adaptive schedule accelerates initial convergence for new cameras while preserving accumulated corrections for returning ones, but multi-session operation is still required for the factor to fully stabilize; none of the 48 accumulated cameras had converged at time of evaluation. The scale factors therefore capture the geometric projection bias present at each camera and should be interpreted as a running estimate rather than a definitive calibration.

Because the system derives all geographic positions from map geometry rather than on-vehicle GPS receivers, localization accuracy is bounded by the quality of the NodeLink

road database and the precision of the perspective projection; errors in map alignment or road-shape approximation propagate directly into vehicle GPS coordinates. Additionally, a camera’s effective field of view is strongly shaped by its mounting angle: a steeply pitched installation sees a narrow, elongated strip of road, producing a thin FOV polygon on the map and limiting the usable observation window even after geometric calibration.

We also state plainly the remaining evaluation gap. We now report quantitative *localization* accuracy against an external reference—the satellite-referenced landmark measurements of Section V—but this validation is still limited in scope: two cameras with a handful of landmarks each, bounded below by the 1–3 m precision of the aerial imagery itself, and not yet a large-scale study. We do *not* yet report quantitative *speed* accuracy against an external reference; the speed figures in Table V confirm plausible output rather than benchmark accuracy. The landmark protocol naturally extends to more cameras and denser landmark sets, and speed can be evaluated on the public Korean-CCTV and BrnoCompSpeed references [20], [21]. We leave this broader quantitative validation—more cameras, and a speed benchmark—as the immediate next step; the contribution claimed here is the map-anchored localization and self-calibrating pipeline, not a speed-accuracy result.

A structural limitation is the system’s dependence on national ITS infrastructure: camera streams and segment speeds are fetched from a government-operated open API [13] whose availability and authentication policy are outside the operator’s control and may be restricted or discontinued. Camera placement and viewing angle are fixed by road authorities and cannot be adjusted; some installations are positioned too far or too close to capture a useful road segment, and extreme viewpoints cannot be recovered through calibration. Adverse conditions—low illumination, rain, lens contamination, and hardware faults—can suppress the visual signal entirely, leaving the pipeline with no basis for detection or calibration. These constraints are inherent to re-using existing infrastructure without modification.

VII. CONCLUSION

We built a real-time traffic digital twin that localizes vehicles on a map from uncalibrated public CCTV and estimates their speed without ground truth. The key ideas are a road-centreline-following two-stage transform, a robust five-stage speed estimator, and a per-camera scale-correction framework with an ITS-referenced online update loop, wrapped in a resilient dual-path pipeline with congestion analytics and a historical store. The automatic calibration pipeline—geometric pose solver plus the accumulating scale factor—substantially reduces the per-camera manual setup required to deploy across many cameras, while the system remains operational even before the ITS loop converges.

A distinguishing property is that the system requires *no GPS instrumentation on vehicles*: conventional traffic monitoring relies on embedded GPS loggers or dedicated roadside sensors, both of which demand significant infrastructure investment. By re-purposing cameras that road authorities already operate, the system tracks multiple vehicle classes simultaneously across

multiple views with no additional hardware cost. As public CCTV coverage continues to expand and object detectors advance, this class of vision-only traffic twin becomes increasingly practical at scale. With wider network integration, the same localization principle—positioning objects through perspective projection rather than satellite signal—could extend to GPS-denied environments such as tunnels, underground parking, and indoor logistics facilities; the per-camera track-ID scheme further enables cross-camera vehicle re-identification for a range of fleet management and security applications.

The source code is publicly available at <https://github.com/MinsooAhn02/DigitalTwin>.

ACKNOWLEDGMENT

The authors thank Prof. Jihoon Ryoo of SUNY Korea for guidance and feedback throughout CSE327 Computer Vision. This work used the Korea Intelligent Transport Systems (ITS) Open API and the Ministry of Land, Infrastructure and Transport (MOCT) NodeLink road network dataset, both provided by the Republic of Korea.

REFERENCES

- [1] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” <https://github.com/ultralytics/ultralytics>, 2023, open-source object detection and tracking framework.
- [2] Ultralytics, “YOLO26: Key architectural enhancements and performance benchmarking for real-time object detection,” <https://arxiv.org/abs/2509.25164>, 2025, arXiv:2509.25164.
- [3] NVIDIA, “TensorRT: High-performance deep learning inference optimizer and runtime,” <https://developer.nvidia.com/tensorrt>, 2023.
- [4] M. Broström, “BoxMOT: Pluggable SOTA multi-object tracking modules,” <https://github.com/mikel-brostrom/boxmot>, 2023.
- [5] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-object tracking by associating every detection box,” in *Proc. European Conf. on Computer Vision (ECCV)*, 2022, pp. 1–21.
- [6] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, “BoT-SORT: Robust associations multi-pedestrian tracking,” *arXiv preprint arXiv:2206.14651*, 2022.
- [7] J. Cao, J. Pang, X. Weng, R. Khirrodar, and K. Kitani, “Observation-centric SORT: Rethinking SORT for robust multi-object tracking,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [8] K. Zhou, Y. Yang, A. Cavallaro, and T. Xiang, “Omni-scale feature learning for person re-identification,” *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2019.
- [9] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.
- [10] G. Bradski, “The OpenCV library,” *Dr. Dobbs’s Journal of Software Tools*, 2000.
- [11] M. Dubska, A. Herout, R. Juránek, and J. Sochor, “Fully automatic roadside camera calibration for traffic surveillance,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 3, pp. 1162–1171, 2014.
- [12] Korea Transport Database (KTDB) / Ministry of Land, Infrastructure and Transport, “Standard node-link (MOCT) road network dataset,” <https://www.its.go.kr/nodelink>, 2023.
- [13] National Transport Information Center (ITS), Republic of Korea, “ITS open API: Real-time CCTV and traffic information,” <https://openapi.its.go.kr>, 2023.
- [14] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. 2nd Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.
- [15] A. M. Andrew, “Another efficient algorithm for convex hulls in two dimensions,” *Information Processing Letters*, vol. 9, no. 5, pp. 216–219, 1979.
- [16] Transportation Research Board, *Highway Capacity Manual (HCM)*. Washington, D.C.: National Research Council, 2016.
- [17] vis.gl / Open Visualization, “deck.gl: WebGL2-powered geospatial visualization framework,” <https://deck.gl>, 2023.
- [18] MapLibre Contributors, “MapLibre GL JS,” <https://maplibre.org>, 2023.
- [19] S. Ramírez, “FastAPI,” <https://fastapi.tiangolo.com>, 2018.
- [20] J. Sochor, J. Spánhel, and A. Herout, “Comprehensive data set for automatic single camera visual speed measurement,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 5, pp. 1633–1643, 2019.
- [21] J. Revaud and M. Humenberger, “Robust automatic monocular vehicle speed estimation for traffic surveillance,” in *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2021, pp. 3962–3972.
- [22] M. Krist, J. Sochor, R. Juránek, J. Spánhel, V. Bartl, and A. Herout, “FARSEC: A reproducible framework for automatic real-world speed estimation using cameras,” <https://arxiv.org/abs/2301.03802>, 2023, arXiv:2301.03802.